JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface Deque<E>

Type Parameters:

E - the type of elements held in this deque

All Superinterfaces:

Collection<E>, Iterable<E>, Queue<E>, SequencedCollection<E>

All Known Subinterfaces:

BlockingDeque<E>

All Known Implementing Classes:

ArrayDeque, ConcurrentLinkedDeque, LinkedBlockingDeque, LinkedList

---

```
public interface Deque<E>
extends Queue<E>, SequencedCollection<E>
```

A linear collection that supports element insertion and removal at both ends. The name *deque* is short for "double ended queue" and is usually pronounced "deck". Most Deque implementations place no fixed limits on the number of elements they may contain, but this interface supports capacity-restricted deques as well as those with no fixed size limit.

This interface defines methods to access the elements at both ends of the deque. Methods are provided to insert, remove, and examine the element. Each of these methods exists in two forms: one throws an exception if the operation fails, the other returns a special value (either null or false, depending on the operation). The latter form of the insert operation is designed specifically for use with capacity-restricted Deque implementations; in most implementations, insert operations cannot fail.

The twelve methods described above are summarized in the following table:

Summary of Deque methods

	First Element (Head)		Last Element (Tail)	
	Throws exception	*Special value*	*Throws exception*	*Special value*
Insert	addFirst(e)	offerFirst(e)	addLast(e)	offerLast(e)
Remove	removeFirst()	pollFirst()	removeLast()	pollLast()
Examine	getFirst()	peekFirst()	getLast()	peekLast()

This interface extends the Queue interface. When a deque is used as a queue, FIFO (First-In-First-Out) behavior results. Elements are added at the end of the deque and removed from the beginning. The methods inherited from the Queue interface are precisely equivalent to Deque methods as indicated in the following table:

Comparison of Queue and Deque methods

Queue Method	Equivalent Deque Method
`add(e)`	`addLast(e)`
`offer(e)`	`offerLast(e)`
`remove()`	`removeFirst()`
`poll()`	`pollFirst()`
`element()`	`getFirst()`
`peek()`	`peekFirst()`

Deques can also be used as LIFO (Last-In-First-Out) stacks. This interface should be used in preference to the legacy `Stack` class. When a deque is used as a stack, elements are pushed and popped from the beginning of the deque. Stack methods are equivalent to `Deque` methods as indicated in the table below:

Comparison of Stack and Deque methods

Stack Method	Equivalent Deque Method
`push(e)`	`addFirst(e)`
`pop()`	`removeFirst()`
`peek()`	`peekFirst()`

Note that the `peek` method works equally well when a deque is used as a queue or
a stack; in either case, elements are drawn from the beginning of the deque.

This interface provides two methods to remove interior elements, `removeFirstOccurrence` and `removeLastOccurrence`.

Unlike the `List` interface, this interface does not provide support for indexed access to elements.

While `Deque` implementations are not strictly required to prohibit the insertion of null elements, they are strongly encouraged to do so. Users of any `Deque` implementations that do allow null elements are strongly encouraged *not* to take advantage of the ability to insert nulls. This is so because `null` is used as a special return value by various methods to indicate that the deque is empty.

`Deque` implementations generally do not define element-based versions of the `equals` and `hashCode` methods, but instead inherit the identity-based versions from class `Object`.

This interface is a member of the Java Collections Framework.

Since:

1.6

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
boolean	**add**(E e)	Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque) if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.
boolean	**addAll**(Collection<? extends E> c)	Adds all of the elements in the specified collection at the end of this deque, as if by calling `addLast(E)` on each one, in the order that they are returned by the collection's iterator.
void	**addFirst**(E e)	Inserts the specified element at the front of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
void	**addLast**(E e)	

		Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
`boolean`	**`contains`**`(Object o)`	Returns `true` if this deque contains the specified element.
`Iterator<E>`	**`descendingIterator`**`()`	Returns an iterator over the elements in this deque in reverse sequential order.
`E`	**`element`**`()`	Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque).
`E`	**`getFirst`**`()`	Retrieves, but does not remove, the first element of this deque.
`E`	**`getLast`**`()`	Retrieves, but does not remove, the last element of this deque.
`Iterator<E>`	**`iterator`**`()`	Returns an iterator over the elements in this deque in proper sequence.
`boolean`	**`offer`**`(E e)`	Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque)

		if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and `false` if no space is currently available.
`boolean`	**`offerFirst`**`(E e)`	Inserts the specified element at the front of this deque unless it would violate capacity restrictions.
`boolean`	**`offerLast`**`(E e)`	Inserts the specified element at the end of this deque unless it would violate capacity restrictions.
`E`	**`peek`**`()`	Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.
`E`	**`peekFirst`**`()`	Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.
`E`	**`peekLast`**`()`	Retrieves, but does not remove, the last element of this deque, or returns

		`null` if this deque is empty.
`E`	**`poll`**`()`	Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.
`E`	**`pollFirst`**`()`	Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.
`E`	**`pollLast`**`()`	Retrieves and removes the last element of this deque, or returns `null` if this deque is empty.
`E`	**`pop`**`()`	Pops an element from the stack represented by this deque.
`void`	**`push`**`(E e)`	Pushes an element onto the stack represented by this deque (in other words, at the head of this deque) if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
`E`	**`remove`**`()`	Retrieves and removes the head of the queue represented by this deque (in other words,

		the first element of this deque).
boolean	**remove**(Object o)	Removes the first occurrence of the specified element from this deque.
E	**removeFirst**()	Retrieves and removes the first element of this deque.
boolean	**removeFirstOccurrence**(Object o)	Removes the first occurrence of the specified element from this deque.
E	**removeLast**()	Retrieves and removes the last element of this deque.
boolean	**removeLastOccurrence**(Object o)	Removes the last occurrence of the specified element from this deque.
default Deque<E>	**reversed**()	Returns a reverse-ordered view of this collection.
int	**size**()	Returns the number of elements in this deque.

Methods declared in interface Collection

clear, containsAll, equals, hashCode, isEmpty, parallelStream, removeAll, removeIf, retainAll, spliterator, stream, toArray, toArray, toArray

Methods declared in interface Iterable

`forEach`

Method Details

addFirst

`void addFirst(E e)`

Inserts the specified element at the front of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use method `offerFirst(E)`.

Specified by:

`addFirst` in interface `SequencedCollection<E>`

Parameters:

e - the element to add

Throws:

`IllegalStateException` - if the element cannot be added at this time due to capacity restrictions

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

addLast

`void addLast(E e)`

Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use method `offerLast(E)`.

This method is equivalent to `add(E)`.

Specified by:

`addLast` in interface `SequencedCollection<E>`

Parameters:

e - the element to add

Throws:

`IllegalStateException` - if the element cannot be added at this time due to capacity restrictions

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

offerFirst

```
boolean offerFirst(E e)
```

Inserts the specified element at the front of this deque unless it would violate capacity restrictions. When using a capacity-restricted deque, this method is generally preferable to the `addFirst(E)` method, which can fail to insert an element only by throwing an exception.

Parameters:

e - the element to add

Returns:

`true` if the element was added to this deque, else `false`

Throws:

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

offerLast

```
boolean offerLast(E e)
```

Inserts the specified element at the end of this deque unless it would violate capacity restrictions. When using a capacity-restricted deque, this method is generally preferable to the `addLast(E)` method, which can fail to insert an element only by throwing an exception.

Parameters:

e - the element to add

Returns:

`true` if the element was added to this deque, else `false`

Throws:

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

removeFirst

`E removeFirst()`

Retrieves and removes the first element of this deque. This method differs from `pollFirst` only in that it throws an exception if this deque is empty.

Specified by:

`removeFirst` in interface `SequencedCollection<E>`

Returns:

the head of this deque

Throws:

`NoSuchElementException` - if this deque is empty

removeLast

`E removeLast()`

Retrieves and removes the last element of this deque. This method differs from `pollLast` only in that it throws an exception if this deque is empty.

Specified by:

`removeLast` in interface `SequencedCollection<E>`

Returns:

the tail of this deque

Throws:

`NoSuchElementException` - if this deque is empty

pollFirst

`E pollFirst()`

Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.

Returns:

the head of this deque, or `null` if this deque is empty

pollLast

`E pollLast()`

Retrieves and removes the last element of this deque, or returns `null` if this deque is empty.

Returns:

the tail of this deque, or `null` if this deque is empty

getFirst

`E getFirst()`

Retrieves, but does not remove, the first element of this deque. This method differs from `peekFirst` only in that it throws an exception if this deque is empty.

Specified by:

`getFirst` in interface `SequencedCollection<E>`

Returns:

the head of this deque

Throws:

`NoSuchElementException` - if this deque is empty

getLast

`E getLast()`

Retrieves, but does not remove, the last element of this deque. This method differs from `peekLast` only in that it throws an exception if this deque is empty.

Specified by:

`getLast` in interface `SequencedCollection<E>`

Returns:

the tail of this deque

Throws:

`NoSuchElementException` - if this deque is empty

peekFirst

`E peekFirst()`

Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.

Returns:

the head of this deque, or `null` if this deque is empty

peekLast

`E peekLast()`

Retrieves, but does not remove, the last element of this deque, or returns `null` if this deque is empty.

Returns:

the tail of this deque, or `null` if this deque is empty

removeFirstOccurrence

`boolean removeFirstOccurrence(Object o)`

Removes the first occurrence of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes the first element `e` such that `Objects.equals(o, e)` (if such an element exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if an element was removed as a result of this call

Throws:

`ClassCastException` - if the class of the specified element is incompatible with this deque (optional)

`NullPointerException` - if the specified element is null and this deque does not permit null elements (optional)

removeLastOccurrence

`boolean removeLastOccurrence(Object o)`

Removes the last occurrence of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes the last element `e` such that `Objects.equals(o, e)` (if such an element

exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if an element was removed as a result of this call

Throws:

`ClassCastException` - if the class of the specified element is incompatible with this deque (optional)

`NullPointerException` - if the specified element is null and this deque does not permit null elements (optional)

add

```
boolean add(E e)
```

Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque) if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use `offer`.

This method is equivalent to `addLast(E)`.

Specified by:

`add` in interface `Collection<E>`

Specified by:

`add` in interface `Queue<E>`

Parameters:

`e` - the element to add

Returns:

`true` (as specified by `Collection.add(E)`)

Throws:

`IllegalStateException` - if the element cannot be added at this time due to capacity restrictions

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

offer

```
boolean offer(E e)
```

Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque) if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and `false` if no space is currently available. When using a capacity-restricted deque, this method is generally preferable to the `add(E)` method, which can fail to insert an element only by throwing an exception.

This method is equivalent to `offerLast(E)`.

Specified by:

`offer` in interface `Queue<E>`

Parameters:

`e` - the element to add

Returns:

`true` if the element was added to this deque, else `false`

Throws:

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

remove

```
E remove()
```

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque). This method differs from `poll()` only in that it throws an exception if this deque is empty.

This method is equivalent to `removeFirst()`.

Specified by:

`remove` in interface `Queue<E>`

Returns:

the head of the queue represented by this deque

Throws:

`NoSuchElementException` - if this deque is empty

poll

`E poll()`

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `pollFirst()`.

Specified by:

`poll` in interface `Queue<E>`

Returns:

the first element of this deque, or `null` if this deque is empty

element

`E element()`

Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque). This method differs from `peek` only in that it throws an exception if this deque is empty.

This method is equivalent to `getFirst()`.

Specified by:

`element` in interface `Queue<E>`

Returns:

the head of the queue represented by this deque

Throws:

`NoSuchElementException` - if this deque is empty

peek

`E peek()`

Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `peekFirst()`.

Specified by:

`peek` in interface `Queue<E>`

Returns:

the head of the queue represented by this deque, or `null` if this deque is empty

addAll

```
boolean addAll(Collection<? extends E> c)
```

Adds all of the elements in the specified collection at the end of this deque, as if by calling `addLast(E)` on each one, in the order that they are returned by the collection's iterator.

When using a capacity-restricted deque, it is generally preferable to call `offer` separately on each element.

An exception encountered while trying to add an element may result in only some of the elements having been successfully added when the associated exception is thrown.

Specified by:

`addAll` in interface `Collection<E>`

Parameters:

`c` - the elements to be inserted into this deque

Returns:

`true` if this deque changed as a result of the call

Throws:

`IllegalStateException` - if not all the elements can be added at this time due to insertion restrictions

`ClassCastException` - if the class of an element of the specified collection prevents it from being added to this deque

`NullPointerException` - if the specified collection contains a null element and this deque does not permit null elements, or if the specified collection is null

`IllegalArgumentException` - if some property of an element of the specified collection prevents it from being added to this deque

See Also:

`Collection.add(Object)`

push

```
void push(E e)
```

Pushes an element onto the stack represented by this deque (in other words, at the head of this deque) if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.

This method is equivalent to `addFirst(E)`.

Parameters:

`e` - the element to push

Throws:

`IllegalStateException` - if the element cannot be added at this time due to capacity restrictions

`ClassCastException` - if the class of the specified element prevents it from being added to this deque

`NullPointerException` - if the specified element is null and this deque does not permit null elements

`IllegalArgumentException` - if some property of the specified element prevents it from being added to this deque

pop

```
E pop()
```

Pops an element from the stack represented by this deque. In other words, removes and returns the first element of this deque.

This method is equivalent to `removeFirst()`.

Returns:

the element at the front of this deque (which is the top of the stack represented by this deque)

Throws:

`NoSuchElementException` - if this deque is empty

remove

```
boolean remove(Object o)
```

Removes the first occurrence of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes the first element `e` such that `Objects.equals(o, e)` (if such an element exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

This method is equivalent to `removeFirstOccurrence(Object)`.

Specified by:

`remove` in interface `Collection<E>`

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if an element was removed as a result of this call

Throws:

`ClassCastException` - if the class of the specified element is incompatible with this deque (optional)

`NullPointerException` - if the specified element is null and this deque does not permit null elements (optional)

contains

```
boolean contains(Object o)
```

Returns `true` if this deque contains the specified element. More formally, returns `true` if and only if this deque contains at least one element `e` such that `Objects.equals(o, e)`.

Specified by:

`contains` in interface `Collection<E>`

Parameters:

`o` - element whose presence in this deque is to be tested

Returns:

`true` if this deque contains the specified element

Throws:

`ClassCastException` - if the class of the specified element is incompatible with this deque (optional)

`NullPointerException` - if the specified element is null and this deque does not permit null elements (optional)

size

```
int size()
```

Returns the number of elements in this deque.

Specified by:

`size` in interface `Collection<E>`

Returns:

the number of elements in this deque

iterator

`Iterator<E> iterator()`

Returns an iterator over the elements in this deque in proper sequence. The elements will be returned in order from first (head) to last (tail).

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `Iterable<E>`

Returns:

an iterator over the elements in this deque in proper sequence

descendingIterator

`Iterator<E> descendingIterator()`

Returns an iterator over the elements in this deque in reverse sequential order. The elements will be returned in order from last (tail) to first (head).

Returns:

an iterator over the elements in this deque in reverse sequence

reversed

`default Deque<E> reversed()`

Returns a reverse-ordered view of this collection. The encounter order of elements in the returned view is the inverse of the encounter order of elements in this collection. The reverse ordering affects all order-sensitive operations, including those on the view collections of the returned view. If the collection implementation permits modifications to this view, the modifications "write through" to the underlying collection. Changes to the underlying collection might or might not be visible in this reversed view, depending upon the implementation.

Specified by:

`reversed` in interface `SequencedCollection<E>`

Implementation Requirements:

The implementation in this interface returns a reverse-ordered Deque view. The `reversed()` method of the view returns a reference to this Deque. Other operations on the view are implemented via calls to public methods on this Deque. The exact relationship between calls on the view and calls on this Deque is unspecified. However, order-sensitive operations generally behave as if they delegate to the appropriate method with the opposite

orientation. For example, calling `getFirst` on the view might result in a call to `getLast` on this Deque.

Returns:

a reverse-ordered view of this collection, as a `Deque`

Since:

21

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.